UNIVERSITY OF PASSAU

Faculty of Computer Science and Mathematics

Reverse Engineering – Disassembly



Prof. Dr.-Ing. Christian Hammer

WS 2026/27

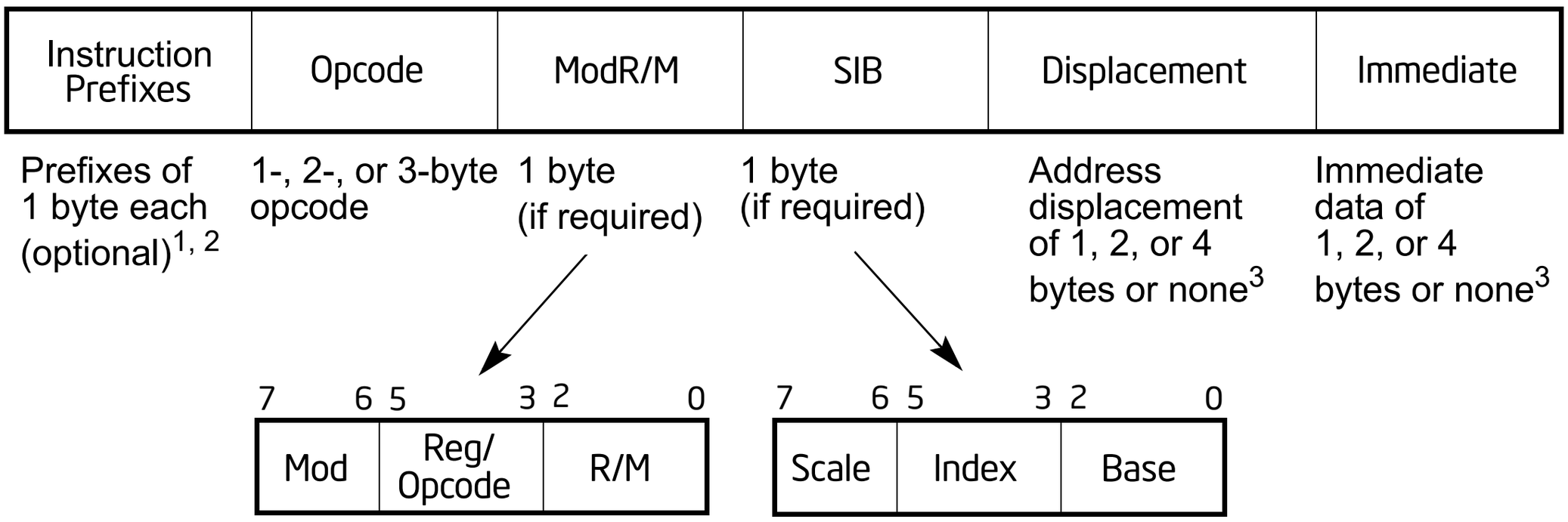
Assembly vs. Disassembly

- Assembler translates instruction mnemonics to machine code
- Builds executable file format containing code and data

- Disassembler converts machine code into instruction mnemonics
- Identifies code and data sections in executable
- Includes labels in code for jump targets
- Other relevant data like string or jump tables

Instruction Decoding

- Large set of instructions with many additional parameter combinations
- Prefixes for instruction lengths, repetition, etc.
- Complex set of rules given in Intel's Architectures Software Developer's Manual (Volume 2), ~200 pages



1. The REX prefix is optional, but if used must be immediately before the opcode; see Section 2.2.1, "REX Prefixes" for additional information.
2. For VEX encoding information, see Section 2.3, "Intel® Advanced Vector Extensions (Intel® AVX)".
3. Some rare instructions can take an 8B immediate or 8B displacement.

- Writing your own decoder is tedious and error-prone
- Open source decoders available for several ISAs
- Popular: Capstone

Linear Sweep Disassembly

- Initialize PC to entry point
- Loop: Decode opcode at PC (length: k bytes)
- Increment PC by k
- Jump to Loop

- Simple and fast
- Used by many debuggers, as compilers (clang, gcc) don’t mix code and data

- Only works if *no data is included into stream of opcodes*

- mov eax, 0x234
- jmp label
- db E0
- label: xor eax, eax
- ret

- b8 34 02 00 00
- eb 01
- e0
- 31 c0
- c3

- mov eax, 0x234
- jmp 0x8
- loopne 0x3a
- db 0xc0
- ret

- Re-start disassembly at jump target (and fall-through for cond. jump)
- Analyze jump tables for targets
- Guarantees proper alignment of actual instructions
- Problem: indirect jump targets (computed at runtime)
- In general undecidable ❌
- Heuristics for function headers (function prolog), jump tables, etc.
- Used in “commercial” disassemblers (Ghidra, IDA Pro, etc.), potential for manual intervention
- May take very long time based on heuristics and analyses employed

Control Flow Obfuscation

- Attempt to obfuscate actual sequence of opcodes
- Jump “into” opcode (within 15 byte code length)
- Potential solution: Start disassembly from any offset
- Invalid instruction found: disregard that offset

- *Semantics preserving transformation* to protect program against reverse engineering efforts
- Prevent or blow up static analysis efforts
- Preclude manual code understanding with debuggers, etc.
- Theoretically impossible to provide perfect obfuscation
- Practical solutions available for certain classes of problems or with reduced strength

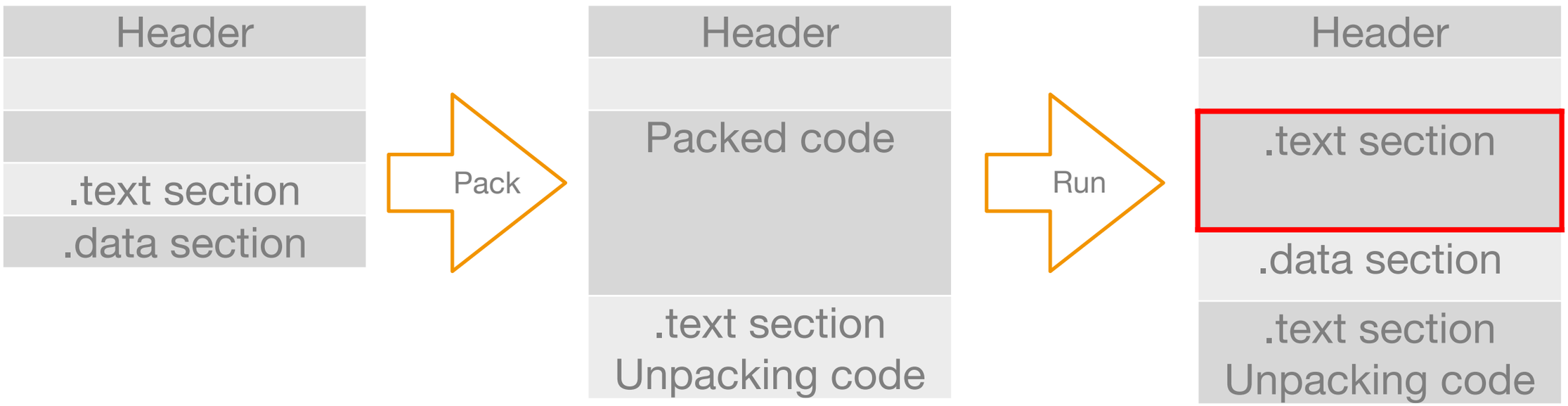
- Modifying/packing/encrypting actual code, reverse transformation at runtime
- (Actual) code is not available in binary for disassembly, only at runtime

- To protect intellectual property (e.g. secret algorithm, copy protection)
- To hide application secrets (“magic keys” for special APIs, decryption keys e.g. for media decoding)
- To hide malicious behavior

- Code generated at runtime is invisible to reverse engineering efforts
- Dynamic code loading
  - Code is (down-)loaded at runtime and integrated in memory
  - Requires dumping of memory image before reverse engineering
- Self-modifying code
  - Code modified at runtime, potentially multiple times between executions of same functionality
  - Manual effort required, unsetting W XOR X property of code
- Packing/Encryption
  - Popular with malware, defeats simple pattern detection
  - Free and commercial implementations available

Packing Executable

- Code packed/encrypted after linking
- Load time: unpacking routine (visible to disassembler)



- Decrypt code/data etc. into sections
- “Manually” resolve imports, duplicate OS loader’s functionality
- Jump to original entry point (tail jump)
  may obfuscate transfer (call, ret, intricate transfer variants)

- For static analysis of packed code
- Reverse the unpacking stub and include its functionality into disassembly routine
- Automatic static unpacking programs exist for certain tools

- Run program until unpacker is done and dump memory to disassemble
- Automated dynamic unpacking (ongoing research)
  - Detects tail call, dumps and disassembles
  - Rediscover imports

- Dynamic analysis potentially thwarted by anti-debugging, anti-VM
- Unpacking on finer granularity (by function), no single memory dump

- Metamorphic Malware to prevent pattern-matching scanners
- Changes syntax of code (e.g. replace by semantically equivalent opcodes)
- Often rendered useless by automated static analysis (compiler techniques)
- Example: Jump insertion
- Can lead to disassembly errors in linear sweep
- Recursive traversal undoes effect

- Obfuscation of control flow to prevent building an accurate CFG
- Hinders manual/automatic understanding of code
- Examples
  - Function in-/outlining
  - OS-based control indirection (exception handlers, longjmp)
  - Opaque predicates
  - Function joining
  - Control flow flattening
  - Virtualization

Opaque Predicates

- Insert spurious branches (indirect jump that always takes one direction)
- Blows up CFG with unused jump targets, potential cycles in CFG
- Opaque predicate: (at least at runtime) constant expression, hard to prove constant by analysis
- Often arithmetic theorems (x*x>=0), (x << 1 % 2 == 0)

- Can be detected by pattern matching of given tool
- Brute force all possible input values (2^32 or 2^64)
- SMT (satisfiability modulo solvers)
- Dynamic analysis ignoring unused code

Function Joining

- Merge two unrelated functions to obfuscate semantics
- Additional argument selects function
- Total size of actual arguments should match to avoid easy detection
- Undefined behavior (in C), in Assembly no cast necessary

```c
int foo(int x, char *z) {
  // do foo
  return x * 2;
}

char bar(char *f, char *z) {
  // do bar
  return f[0];
}
```

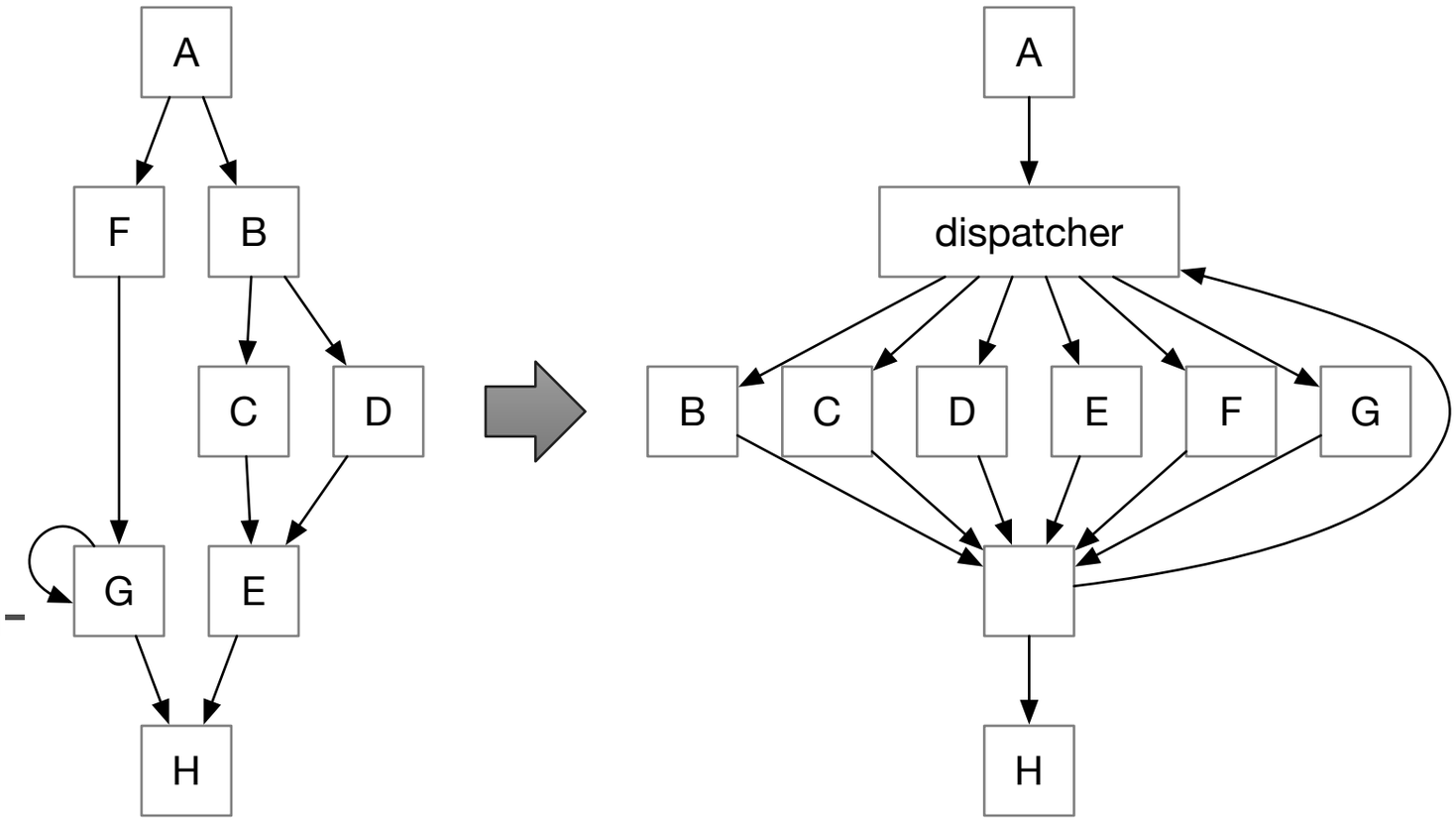


```c
void *foobar(int t, void *a, void *b) {
  if (t > 0) {
    // do foo
      return (int)a * 2;
  } else {
      // do bar
      return ((char *)a)[0];
  }
}
```

- Counter-measures:
- Split according to selector argument
- Select function to call at call site based on selector value
- Potentially remove cast at call site

Control Flow Flattening

- Destroy Control Flow Graph almost completely
- Basic blocks are numbered
- Add dispatcher (switch statement jumping to given basic block number)
- All control flow instructions replaced by jump to dispatch-er with successor number
- Can be combined with spurious basic blocks and/or dead code paths



- Generalized version of control flow flattening
- Use virtual machine to execute instructions
- One of the strongest obfuscation schemes
- Commercial tools
- Opcode with (e.g. two) operand numbers stored in code array
- Virtual program counter used to move in code array
- Operands are pointers to virtual data array

- Identify virtual program counter (vpc)
- Statically execute program, computing the vpc (symbolic execution)
- Clone each block for each opcode at vpc
- Connect blocks based on vpc successors
- Use compiler optimizations to clean up code
  (constant propagation, dead code elimination, jump threading)

- Constant unfolding (replace constant by computation)
- Data-encoding schemes (e.g. homomorphic encryption)
- Dead code insertion
- Arithmetic substitution via identities (e.g. -x = ~x + 1)
- Mixed boolean-arithmetic (MBA) expressions
  $(x_1 \,\&\&\, x_2) + (x_1 \,\&\&\, !x_2) + (!x_1) + (x_3 \,||\, x_4)$
  Only depends on last term!
- Pattern-based obfuscation
  (map instructions into more complicated sequence of instructions)

Summary

- Decoding tedious but straight-forward
- Disassembly needs knowledge of function entry points for correct synchronization (variable-length opcodes)
- Several obfuscation techniques exist to increase obscurity (no formal guarantees)
- Combination of obfuscation techniques increases cost of program analysis superlinearly
- Disadvantage of obfuscation: Slow down, thus often only for sensitive parts

- B. Dang et al.: Practical Reverse Engineering, Wiley 2014
- J. Kinder: Reverse Engineering, Lecture at UBW, 2020
- Andrew Griffiths: Protecting binaries, 2005
- Andrew Griffiths: Binary protection schemes